

All Chapter One-Pagers

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

All Chapter One-Pagers

Contents

Chapter 1 One-Pager — Reinforcement Learning Basics	3
Chapter 2 One-Pager — Game Theory and CFR Basics	4
Chapter 3 One-Pager — CFR Variants and Monte Carlo Methods	5
Chapter 4 One-Pager — Game Abstraction and Scaling	6
Chapter 5 One-Pager — Neural Networks for Imperfect-Information Games	7
Chapter 6 One-Pager — End-to-End Game AI Architectures	8
Chapter 7 One-Pager — Opponent Modeling	9
Chapter 8 One-Pager — Safe Exploitation	10
Chapter 9 One-Pager — Multi-Agent Reinforcement Learning	11
Chapter 10 One-Pager — Population-Based Training and Evolutionary Game Theory	12
Chapter 11 One-Pager — Coalition Formation and Detection	13
Chapter 12 One-Pager — Sequence Models and LLM Agents in Strategic Settings	14

Chapter 1 One-Pager — Reinforcement Learning Basics

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

April 2026

Problem. Everything later in this plan — neural equilibrium finding (Chapter 5), the opponent-model actuator (Chapters 7-8), multi-agent learning (Chapter 9) — is assembled out of value networks, policy gradients and experience replay. Chapter 1 is the foundation layer: how an agent improves a policy from reward alone, in the fully observable single-agent setting, before that picture is complicated by a second, hidden, adversarial player. It feeds Contribution #1 indirectly (the architecture patterns are reused by the adaptive opponent model) and directly supplies the implementation vocabulary every later chapter assumes.

Approach. One algorithm from each family, written from scratch in PyTorch: **DQN** (value-based, off-policy) on **CartPole-v1** — Q-network [4->128->128->2], circular replay buffer, frozen target network synced every 5 episodes — and **PPO** (policy-gradient, on-policy) on **LunarLander-v3** — separate actor [8->128->128->4] and critic, GAE ($\lambda=0.95$) by reverse sweep, clipped surrogate at $\text{eps}=0.2$. Each was then benchmarked against its **Stable-Baselines3** counterpart (PPO at matched core hyperparameters and the same 500K-step budget; DQN against the RL Zoo’s tuned CartPole settings at 100K steps, run of 6 April 2026), scored on best rolling-100 average so that early stopping cannot flatter the result. **All numbers below are measured.**

Key results (measured).

- *Both targets met.* DQN solves CartPole at **episode 1011**, 100-episode average **477.5** (target 475); PPO solves LunarLander at **264,192 steps** at **202.2** (target 200) — 236K steps inside its budget.
- *The from-scratch versions scored higher than the SB3 runs on these tasks.* Best rolling-100: DQN **477.5 vs 222.4**, PPO **203.6 vs 131.2**. The causes are untested: each side was run once (ours without a fixed seed), and the SB3 DQN, on the Zoo’s settings, fell short at twice the Zoo’s 50K-step budget, which needs a second seed before it is read as a property of SB3.
- *eps_min was the decisive DQN knob.* At 0.01 the run peaked at **479.1** (episode 810) and decayed to **302.6**; 0.001 plus best-model checkpointing turned a degrading policy into a stable one.
- *Capacity, not budget, was PPO’s bottleneck.* [64, 64] peaked at **179.9** and never crossed 200; [128, 128] cleared it. Advantage normalisation was kept throughout; rewards span -500 to +300, and without the 0.01 entropy bonus the policy collapsed to hovering in place.
- *Honest read on the comparison.* SB3’s PPO was still climbing at 500K and would likely reach target given 1-2M steps, and the DQN baseline is a single run with tuned settings. The claim is a task-specific advantage, not a better algorithm.

Thesis connection. The 300-500-line stack (against SB3’s ~10K) is the deliberate price paid for surgical modifiability: injecting belief-state observations into an actor-critic is a deep subclassing exercise in SB3 and a local edit here, which is exactly what the exploitation work in Chapters 7-8 needs. The same actor-critic pair returns as MAPPO/MADDPG in Chapter 9, and the value-network machinery returns as Deep CFR’s advantage network in Chapter 5.

Open questions. The MDP assumption that makes all of this work — a fully observable state — is precisely what poker breaks: over an information set, an action’s value depends on the opponent’s strategy, which keeps changing while it learns, and DQN’s greedy **max** cannot express the mixed strategies equilibrium play needs, which is why Chapter 2 changes tool rather than scale. Open: how much of the single-agent stability toolkit (target networks, trust regions, advantage normalisation) survives when the environment is itself a learning agent (deferred to Chapter 9’s non-stationarity), and whether the sample-efficiency edge measured here is a real property or an artifact of two small, well-conditioned control tasks.

Chapter 2 One-Pager — Game Theory and CFR Basics

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

April 2026

Problem. Chapter 1’s machinery assumes a fully observable state and a stationary environment; poker has neither. Two-player zero-sum imperfect-information games need a different solution concept — Nash equilibrium defined over *information sets* — and a different quality measure: exploitability, not reward. Chapter 2 establishes both, plus the algorithm that computes them, because everything after it is either an approximation of CFR (Chapters 3-5), a deliberate departure from the equilibrium CFR produces (Chapters 7-8), or an attempt to say what equilibrium even means once $N > 2$ (Chapter 9, 11).

Approach. Chance-sampled CFR written from scratch on **Kuhn Poker** — 3 cards, 2 players, 12 information sets: small enough to have a closed-form equilibrium *family* (parameterised by **alpha** in $[0, 1/3]$), rich enough to contain bluffing, mixed strategies and indifference. Hand-coded components: the game engine, recursive counterfactual traversal with regret matching and chance sampling, and an exact exploitability evaluator that enumerates all $2^6 = 64$ pure strategies — because a per-state “oracle” best response is *not* a best response; the best-responder must commit to one action per information set. Trained for 100,000 iterations (seed 0), with an OpenSpiel cross-verification script alongside. **All numbers below are measured** (implementation/step02/models/cfr_results.json).

Key results (measured).

- *CFR lands close to the analytical equilibrium family.* The recovered bluff parameter is **alpha** = 0.1792, and the family’s own internal constraint $P(\text{bet} \mid K) = 3 \cdot \text{alpha}$ holds to within **0.038** (measured **0.5758** against the **0.5377** its own alpha implies); even so, the exploitability of the whole profile is only **0.006**.
- *Pure decisions converge hard; mixed frequencies carry the $O(1/\sqrt{T})$ tail.* King bets and calls at $\geq$ **0.9999**, Jack folds to a bet at **0.99996**, Queen passes at the root at **0.9995** — while the mixing sits **0.005-0.038** off the closed form (Jack bluff after a pass **0.3387** vs $1/3$; Queen call **0.5307** vs $1/3 + \text{alpha} = 0.5126$). The frequencies are the slow part, which is precisely where the residual regret lives.
- *Game value* of the average strategy **-0.0555** against the exact **-1/18 = -0.0556** (the running mean of sampled payoffs over all 100k iterations is **-0.0613**), reproducing Player 0’s structural first-mover disadvantage.
- *The convergence rate is consistent with the theoretical bound.* Log-log exploitability slope **-0.52 $\pm$ 0.02** (5 seeds, 100-100k iterations) against the **-0.5** of the $O(1/\sqrt{T})$ bound.
- *Game value alone is not a validity check.* A strategy that always bluffs the Jack can still average near $-1/18$ while being trivially exploitable; only exploitability catches it. That is why exploitability — not reward — is the metric carried forward into Chapters 7-8 and into the evaluation methodology (Contribution #3).

Thesis connection. Nash is the baseline this thesis exists to leave: Contribution #1 reads a specific opponent in order to justify departing from it, and Contribution #2 bounds how far the departure may go. Concretely, the exact best-response and exploitability code written here becomes the literal oracle reused in Chapters 7-10, and Kuhn remains the smallest testbed where every later claim can be bracketed by an exact answer.

Open questions. Why the *average* strategy converges when the current strategy does not — the intuition (averaging damps the overshoot, as Polyak averaging does) is not a proof. Everything here rests on the two-player zero-sum minimax theorem, and neither CFR’s guarantee nor exploitability’s meaning survives into N-player or general-sum settings (Chapter 9, 11). And even with one sampled deal per iteration, CFR traverses every action sequence of that deal, so this exact algorithm is already out of budget one game up — the motivation for Chapter 3’s Monte Carlo sampling and Chapter 4’s abstraction.

Chapter 3 One-Pager — CFR Variants and Monte Carlo Methods

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

April 2026

Problem. Chapter 2’s vanilla CFR visits every information set on every iteration, which stops being affordable one game up from Kuhn. The field’s two answers are a better-constant full-traversal method (**CFR+**) and *sampling* the tree (**MCCFR**), and the received wisdom is that sampling is what made real poker solvable. Chapter 3 asks which one actually wins, on what size of game, and why — the question that decides the solver every later chapter depends on.

Approach. All four algorithms hand-coded in Python + NumPy on **Leduc Poker** (6 cards, two betting rounds, a revealed community card, 936 information sets, 10,200 nodes, 120 deals): vanilla CFR, CFR+ (regret flooring, linear averaging, alternating updates), MCCFR **external sampling** (all traverser actions, one sampled opponent action), and MCCFR **outcome sampling** (one root-to-terminal trajectory, $\text{eps} = 0.6$ on-policy mixture with importance-sampling correction). Graded by an exact information-set-constrained best response, under a common **180-second wall-clock budget**, with OpenSpiel as the reference solver. **All numbers below are measured.**

Key results (measured).

- *CFR+ is the workhorse, and it is not close.* At essentially the same iteration count as vanilla CFR (**3,706 vs 3,713**), CFR+ reaches **2.6e-5** exploitability against vanilla’s **4.4e-3** — roughly **170x** better, from two localised code changes (regret flooring and linear averaging; the vanilla baseline already alternates updates).
- *MCCFR loses badly at this game size.* External sampling reaches **5.5e-2** after **3.5M** iterations and outcome sampling **1.0e-1** after **8.3M** — three to four orders of magnitude worse than CFR+ (and 12–23x worse than vanilla CFR) despite ~1,000x more iterations.
- *The reason is variance, not speed, and it is quantified.* Sampling is **945x / 2,228x** cheaper per iteration but needs ~**99,000x / ~520,000x** more of them. Netted into a wall-clock constant $C_w = C/\text{sqrt}(\text{speed})$: **0.075** (vanilla) against **0.767** and **1.143** — at equal time sampling ends **10.2x / 15.3x** less accurate, so reaching a given accuracy takes ~**105x / ~233x** longer.
- *The crossover is derivable and Leduc is far below it.* Sampling breaks even at roughly **1.1M nodes** (external) and **2.4M** (outcome); Leduc’s 10,200 nodes are **105x / 233x** too small, while limit Hold’em (~3e17 states) is far past it by the model, yet was solved by full-traversal CFR+, so the threshold is an order-of-magnitude guide. This reconciles “MCCFR was essential for real poker” with “MCCFR loses here” — same algorithm, opposite side of the line.
- *Cross-validated against OpenSpiel* at a matched 500-iteration budget: vanilla **0.020 vs 0.022**, CFR+ **8.6e-4 vs 9.4e-4**.
- *Vanilla CFR beats its own worst-case rate.* The supposedly constant $\text{eps} \cdot \text{sqrt}(T)$ falls from **0.90** ($T=100$) to **0.27** ($T=3,700$) instead of holding flat, so the $O(1/\text{sqrt}(T))$ upper bound is far from tight here, as is common for a worst-case bound.

Thesis connection. This chapter fixes the tooling for everything that follows: Leduc plus the exact best-response evaluator become the standard testbed of Chapters 7-12, and CFR+ becomes the Nash reference against which every later opponent-modelling, safe-exploitation and LLM number is bracketed. The exactness-versus-variance trade-off measured here returns in Chapter 5, where the sampled estimates feed a network instead of a table.

Open questions. Whether the crossover node count is a property of the algorithms or of this implementation’s constant factors — a vectorised or compiled full-traversal solver shifts **speed_v** and moves the threshold, so the number is honest for this code and not yet a claim about the algorithms in general. Which sampling scheme survives once the strategy is a neural network rather than a table (Chapter 5). And why vanilla CFR converges faster than its worst case.

Chapter 4 One-Pager — Game Abstraction and Scaling

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

May 2026

Problem. Chapter 3 established that a solver’s reach is bounded by tree size, so the classical route to real poker is not a better solver but a *smaller game*: shrink it, solve the shrunken version, translate the strategy back, and pay an exploitability tax for the distinctions you threw away. Chapter 4 measures that tax. The question is not “does abstraction help” but which axis of compression is safe, and whether the compute a reduction buys ever repays the strategic information it destroys.

Approach. A Leduc-family pipeline that compresses along three axes and evaluates every resulting strategy **in its own full game: lossless suit isomorphism** (Leduc payoffs depend only on rank and on whether the private card pairs the board, so suit-isomorphic information sets merge with no loss), **lossy card bucketing** (hand-strength features, EMD-style distances, k-means, in perfect- and imperfect-recall regimes), and **action abstraction plus translation** (nearest-action, probability-split, pseudo-harmonic) on a variable-bet Mini-NL Leduc. Benchmarked with CFR+ under a common **180-second** budget, 3 seeds, on fixed-limit Leduc, Mini-NL Leduc and a 4-rank Extended Leduc. **All numbers below are measured.**

Key results (measured).

- *Lossless compression is the highest-value move, and it wins by buying iterations.* Suit isomorphism cuts fixed-limit Leduc from **936 to 288** information sets and reaches **3.13e-6** exploitability against full CFR+’s **4.44e-5** in the same wall clock — because it completes **14,185** iterations instead of **2,655**. Confirmed at scale on Extended Leduc: **10,304 -> 2,968** info sets (-71%), **0.0272 -> 0.00126**, roughly 6x the iterations.
- *Lossy card buckets install a floor that compute cannot lift.* **k2 0.571, k3 0.382, k5 0.382** — all at 11,000+ iterations, i.e. 4x more solving than the full game and four orders of magnitude worse. The error is in the abstraction, not the budget.
- *Action abstraction, as deployed here, is the dangerous axis.* Mini-NL: the full 4,704-info-set game reaches **0.00692**, while the 936-info-set action abstraction reaches **0.673** despite 5x the iterations — roughly **100x worse** for a 5x smaller tree. Extended Leduc is starker still: suit+action **4.696** and suit+action+buckets **4.734**, both far worse than simply solving the unabstrated game. Under the current deployment rule the mapping error dominates the total (see limitations).
- *A measured artifact, not a law.* **k5** matches **k3** exactly because Leduc’s post-flop hand-strength distributions collapse into three effective shapes; the bucket-count ordering seen here is a property of this tiny game.
- *Stated limitations.* CFR+ is deterministic, so the three seeds are not independent stochastic runs; the action-abstraction numbers measure the current deployment rule (abstract small bets are played as the large bet; the three translators never engage and return identical values), not translation quality, and should be read as a diagnostic failure mode; and the OpenSpiel comparison aligns all 936 information sets as a sanity check, not as proof of identical solver dynamics.

Thesis connection. The reporting format is the transferable part: strategy quality is meaningless without game size beside it, and the Pareto frontier of the two is an early seed of Contribution #3’s evaluation methodology. The mechanism transfers as well — Chapter 7’s type-based opponent modelling is this same “group for tractability” idea moved from state space to strategy space, and its confident-but-wrong failure is action translation’s misspecification in another costume.

Open questions. Static translation is brittle by construction, which points at subgame and nested solving (Chapter 6) as the production-grade answer to off-tree actions. Whether the ordering measured here — card abstraction survivable, action abstraction ruinous — survives a corrected action mapping and holds in a game with enough hand-strength diversity that **k5** genuinely differs from **k3**. And whether an abstraction can be adapted to the opponent rather than fixed in advance, which is where this chapter touches the adaptive framework directly.

Chapter 5 One-Pager — Neural Networks for Imperfect-Information Games

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

May 2026

Problem. Chapters 2-4 all store a strategy as a table indexed by information set, and Chapter 4 showed both escapes from that table — a better solver, a smaller game — running out. The third escape is to stop enumerating: replace the regret table with a function approximator that *generalises* across similar information states, at fixed memory. Chapter 5 studies that substitution — Deep CFR, its single-network variants, NFSP — and what it costs in guarantees.

Approach. Scope, stated honestly: **this chapter stopped after exploration and reading — there is no from-scratch Phase-4 implementation and no implementation report**, and the survey’s closing synthesis is not yet written. The measured work is a two-day exploration against OpenSpiel’s reference solvers on Leduc and Kuhn — Deep CFR at three network sizes, NFSP on both games, tabular MCCFR and CFR+ as baselines — graded by OpenSpiel’s own exploitability so the head-to-head is honest (Chapter 3’s MCCFR runs on a custom engine and is not comparable). **All numbers below are measured** (implementation/step05/exploration/logs/day0{1,2}_results.json; the CFR+ reference run is recorded only in findings.md).

Key results (measured).

- *The chapter’s most actionable artifact is a library bug.* OpenSpiel 1.6.12’s PyTorch DeepCFRSolver **never trains its advantage networks**: if `len(samples.info_state == 0)` takes `len` of an elementwise comparison, always truthy, so it returns before the optimiser step. Advantage losses come back silently `None` and exploitability stays near **~1.69** whatever the iteration count (a uniform random policy scores 2.37). A one-character fix, patched locally; fixed upstream independently in March 2026 (OpenSpiel 1.6.13). *Caveat*: the patched (64,64) run still sits at 1.70, the same level, so its numbers are left out below until a rerun.
- *Where the table fits, the table wins — decisively.* Leduc at comparable wall time: CFR+ **0.00123** (400 iter, 92 s), tabular MCCFR **0.0967** (50k iter, 68 s), Deep CFR **1.14-1.49** at (32,32) and (128,128,128) (120 iter, 87-101 s), NFSP **2.46** (50k episodes) — CFR+ roughly **900x** better than the best Deep CFR run.
- *Those neural numbers are budget artifacts, not verdicts.* Deep CFR’s curves are flat from iteration 30 to 120 with only 40 traversals per iteration (Steinberger 2019 uses 1,500 on Leduc), and OpenSpiel’s own NFSP Leduc example runs **2e7** episodes — **400x** our budget. Rule adopted: distrust an NFSP Leduc curve below $\sim 1e6$ episodes.
- *Smaller network won at this budget.* (32,32) **1.1441** beat (128,128,128) **1.4908** (one seed) — consistent with underfitting on few distinct samples; the ordering probably reverses with more iterations.
- *What under-training looks like inside the policy.* Against a 400-iteration CFR+ reference, a (64,64) Deep CFR at 40 iterations (exploitability 1.70, so provisional until the rerun) has **median total-variation distance 0.35** across 8 sampled Leduc info states (range 0.163-0.708): the direction of each decision is usually right, but the probabilities are pulled toward uniform (**0.73/0.27** where CFR+ is **0.92/0.07**) — MSE regression on noisy counterfactual samples buys an under-confident smoothing of equilibrium.
- *The cost structure that justifies the family anyway.* A Deep CFR outer iteration costs **~0.7 s** against a tabular MCCFR iteration’s **~1.4 ms** ($\sim 500x$), but its per-iteration work is roughly constant in game size while the tabular cost scales with information-set count. Leduc is a teaching benchmark, where neural methods lose to baselines they were never meant to beat.

Thesis connection. Deep CFR’s advantage network is Chapter 1’s value network with a counterfactual target — the joint that lets Chapters 6-8 leave table-sized games behind. More pointedly, the under-confident smoothing measured here is the failure mode Chapter 7 later prices: a model with the right direction and the wrong frequency is what makes best-responding to an imperfect read lose to Nash.

Open questions. The chapter’s own gate is unmet: a hand-coded Deep CFR with a Vitter reservoir sampler, and DREAM’s outcome sampling with baseline subtraction, both remain outstanding, so “can I build it” is unanswered. Untested: whether the tabular-versus-neural ordering flips exactly at Chapter 3’s node-count crossover, and whether the info-state tensor’s structure really matters more than network depth.

Chapter 6 One-Pager — End-to-End Game AI Architectures

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

June 2026

Problem. Chapters 2-5 assembled parts in isolation — CFR variants, abstraction, neural value approximation — without asking how they compose into a system that beats humans, or what such a system guarantees. Chapter 6 surveys the five competition-grade architectures of 2017-2023 as an evolution of design trade-offs rather than a leaderboard: the state of the art this dissertation builds on and departs from.

Approach. A deliberately *architectural* study — what each system computes offline, what it computes in real time, where learning sits, what it can prove — of **DeepStack** (2017), **Libratus** (2017/18), **Pluribus** (2019), **ReBeL** (2020) and **Student of Games** (2023), each scored on the same nine dimensions and placed on three axes: abstraction to neural representation, offline precomputation to real-time search, imperfect-information-only to unified play. **This chapter produced no code; every number below is the result each paper reports, not a measurement of ours.**

Key results (as reported by the papers).

- *The abstraction era was far more exploitable than its match results suggested.* A local-best-response probe showed top competition bots losing **over 3,000 mbb/g** — four times what folding every hand would lose. Closing that gap is what the decade was for.
- *Search at inference, not the offline strategy, carries the edge.* Libratus’s raw blueprint **did not beat** Baby Tartanian8 (-8 ± 15 mbb/g); with nested safe subgame solving on, the same system **won by 63**, and beat four professionals by **147 mbb/g over 120,000 hands**. Solving off-menu bets live also beat nearest-size action translation **119 vs 1,465 mbb/g** worst-case — Chapter 4’s translation failure repaired by refusing to translate.
- *Cost is not the axis it looks like.* DeepStack spent **~175 CPU-core-years** labelling values and Libratus **~25M core-hours**, while Pluribus’s blueprint cost **~12,400 core-hours** — about **\$150** on one server — and still beat five elite professionals at **+48 mbb/g**.
- *Every advance was paid for elsewhere.* Pluribus reached six players by discarding **all** safety guarantees for unsafe search; ReBeL recovered the two-player guarantee and eliminated abstraction and blueprint alike (Dong Kim **+165 mbb/g**) but retreated to two players; Student of Games unified perfect- and imperfect-information play soundly (beating the LBR probe by **+434 mbb/g**) while winning only **2 of 400** Go games against AlphaZero — its authors’ “price of generality”. No system dominates every axis.
- *The one omission all five share.* Each is **opponent-blind by design**: it computes a worst-case-robust strategy and plays it unconditionally. Libratus and Pluribus say so explicitly, and Pluribus does not even know who it is playing.

Thesis connection. That shared omission is the opening this dissertation occupies, and the chapter supplies the machinery to fill it. ReBeL’s **public belief state** is the substrate Contribution #1 widens from beliefs over cards to beliefs over opponent *type*, inheriting the soundness proofs attached to it rather than bolting adaptation on outside. Pluribus is Contribution #2’s argument in one system: if equilibrium buys no safety at $N > 2$ anyway, declining to exploit forfeits a guarantee never held. Exploitability, the LBR probe (repurposed from certificate to stress test) and AIVAT variance reduction are three ready instruments for Contribution #3.

Open questions. Four, straight from the synthesis: opponent-blindness itself (Chapter 7); safe exploitation beyond two-player zero-sum, which Pluribus won without and Student of Games could not extend (Chapter 8); carrying an opponent model *past the depth limit* instead of discarding it at the leaf; and real-time compute budgets, whose six-order-of-magnitude spread binds hardest on an agent that must re-solve *and* re-estimate an opponent at once. Corollary: build on the cheap, open lineage — the flagship code is unreleased or supercomputer-scale.

Chapter 7 One-Pager — Opponent Modeling

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Problem. A Nash-equilibrium strategy is unexploitable but blind: it plays identically against everyone and never punishes a weak opponent. Opponent modeling is the sensor that turns observed actions into an estimate of a specific opponent’s strategy, so the agent can deviate from equilibrium to exploit them — the first half of the thesis’s Behavioral Adaptation Framework (Contribution #1). The hard part: doing so without becoming exploitable yourself.

Approach. Bayesian belief-update loop (prior x likelihood -> posterior -> best response), built behind one shared interface so a single exact best response can be applied to three interchangeable models: (1) **type-based** — a belief over a fixed menu of opponent types; (2) **continuous** — free-form per-situation Dirichlet counts; (3) **consistent** — a sequence-form convex-optimization estimate (Ganzfried 2025). An **adaptive exploiter** runs the observe -> model -> best-respond -> act loop, with a Nash safety blend and an optional change-point detector for non-stationary opponents. Testbeds: Kuhn Poker and Leduc Hold’em, both exactly solvable, so every result is bracketed by exact analytical references.

Key results (measured).

- *Modeling is worth it.* Equilibrium play leaves **0.11-0.28 per hand** on the table against exploitable Kuhn opponents; the gap against a Nash opponent is ~ 0 (you cannot exploit an equilibrium).
- *When the model class fits, best response reaches the exact ceiling* — the type-based model stays within 3% of the best-response ceiling on every opponent in both games (and within two standard errors on nearly all).
- *A confident-but-underfit model makes you exploitable.* On Leduc the continuous model **loses to Nash (-0.175 vs a -0.083 ceiling)** — best-responding to a wrong estimate of an unexploitable opponent opens a leak in its own play. This is the exploitation-vs-safety tension in a single number.
- *Confident-and-wrong is a real risk.* Against an out-of-menu opponent, the type-based posterior commits hard to the nearest representable type; over 300 seeds a wrong type led past hand 100 in $\sim 13\%$ of runs (it always corrects eventually, and never falls once locked).
- *Non-stationarity is scenario-dependent.* Change-point forgetting rescues a harmful stale model (Kuhn: $-0.106 \rightarrow +0.211$) but underperforms simple adaptation when the new opponent is exploitable and the detector false-fires (Leduc). The reaction matters as much as the detection.
- *Consistency.* The consistent model recovers Kuhn strategies accurately (TV $\sim 0.004-0.021$) but its per-update convex solve is too costly for the online loop as built — an empirical answer to the chapter’s real-time-feasibility question, pointing to incremental solving / Chapter 8 approximations.

Thesis connection. Chapter 7 builds the **sensor** (the model); Chapter 8 builds the **actuator** (safe, KL-regularized exploitation). The continuous model’s Nash self-leak is the empirical case for that safety mechanism; the consistency theory is the principled backbone the framework extends.

Open questions. Real-time consistent modeling (incremental convex solve); principled non-stationarity (confidence-scaled forgetting, better change signals); out-of-menu opponents (open-world type discovery); one opponent to many (joint best response is not the combination of individual ones, and the convex guarantee breaks for $N > 2$).

Chapter 8 One-Pager — Safe Exploitation

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Problem. Chapter 7 built the *sensor* — an opponent model — and showed its danger: best-responding to an imperfect model can open a leak in your own play (the continuous model *lost* to Nash on Leduc). Chapter 8 builds the *actuator*: turn a model into profit **without becoming exploitable**. This is the second half of the thesis’s Behavioral Adaptation Framework (Contribution #1) and the launch point for multi-agent safe exploitation (Contribution #2).

Approach. Every safe-exploitation method is the same program — *maximize expected value against the opponent model, subject to a safety floor on your worst-case value* — which is linear in the hero’s sequence-form realization plan. One LP engine solves it by **constraint generation** (solve $\rightarrow$ read policy $\rightarrow$ call an exact best response as the worst-case oracle $\rightarrow$ add a cut if unsafe $\rightarrow$ re-solve), reusing Chapter 7’s engines, best-response, Nash and opponent zoo. Five families differ only in the floor: **RNR** (tunable p), **best equilibrium** (Ganzfried–Sandholm’s baseline; $\geq$ Nash value v^*), **prime-safe** ($\geq v^* - \varepsilon$ for an ε -equilibrium baseline), **SES** (blueprint value, enforced locally on a subgame via a gadget), **adaptation** (no more exploitable than the blueprint). Testbeds: Kuhn and Leduc, both exactly solvable, so every result is bracketed by exact analytical references. **All numbers below are measured.**

Key results (measured).

- *Safe exploitation works on Kuhn, but gains little. Best equilibrium is safe against every opponent* (worst-case $\geq v^*$ within $1e-3$) **and beats Nash’s own EV on every exploitable type** — e.g. **+0.222 vs Nash’s +0.146** against **AlwaysPass** — but only by 0.002–0.076 per hand (1–9 % of the full best response’s gain): a per-hand floor at v^* admits only equilibrium strategies. Full best response earns more (+0.975) but its worst-case collapses to **−0.5** (ruinously exploitable).
- ε *is measured, not fabricated*. Prime-safe/adaptation lower the floor to $v^* - \varepsilon$ with the baseline’s exploitability measured at $\varepsilon = \mathbf{0.0074}$, and earn a little more than best equilibrium by spending that budget (+0.266 vs +0.222 on **AlwaysPass**).
- *Canonical RNR switches bang-bang, not along a smooth curve* (a contradicted prediction). In a game this small the max-min LP jumps from the safe vertex straight to full best response between $p = 0.6$ and 0.7 (sampled in steps of 0.1); mixing the two vertices traces the chord, which beats the naive blend by only ≈ 0.002 here.
- *Headline — within a 40-iteration cap the global solvers did not converge on Leduc; the subgame method (cap 400) did*. On **Leduc**, the global solvers (best equilibrium/prime-safe/adaptation) **fail to converge within a 40-iteration cap** (worst-case **−0.64 to −1.33**, grossly unsafe), while the **subgame method (SES) converges** (194–350 iters) and stays near-safe (worst-case $\approx \mathbf{−0.13}$) while extracting **+0.25 to +0.68** vs weak types. This is the global-vs-local safety gap, measured on a tiny game — but at unequal budgets, so it shows the global loop is slow, not that it cannot converge.
- *The safety guarantee bites against a worst-case adversary, not a benign one*. In the teaching attack (bait $\rightarrow$ Nash reveal), the honest separating signal is the safety-violation count (**full_br 40/40 refits, best equilibrium 0/40**), not realized profit — a gentle Nash “revealer” never claws back full_br’s bait-phase windfall. A punishing test needs an adaptive counter-exploiter.

Thesis connection. Chapter 7 = sensor; Chapter 8 = actuator. The Kuhn results show the actuator behaves as the theory predicts (perfect models, an exactly solvable game); the Leduc non-convergence is the concrete argument for real-time subgame methods (SES / OX-Search) and/or an exact dual-LP formulation, and it is the empirical bridge into the scalable, multi-agent safe exploitation of Contribution #2.

Open questions. Ganzfried–Sandholm’s own gift-risking algorithm (RWYWE) as the stronger two-player baseline — the existing LP with a floor that moves with banked gifts (future work); scalable safety (exact dual-LP vs local/subgame — the Leduc wall, rerun at equal budgets); whether SES is provably safe (its 0.04 residual is measured against v^* ; it holds its own blueprint floor only to the 0.01 tolerance); a punishing teaching attack (adaptive reveal); and **N-player safety** — every guarantee here rests on the two-player zero-sum fact that Nash secures v^* against any opponent, an anchor that vanishes for $N > 2$; the N-player notions that exist are very conservative (team maxmin) or not securable in general (equal share), so exploitation with a baseline-relative loss bound, also against colluding opponents, is open (Contribution #2).

Chapter 9 One-Pager — Multi-Agent Reinforcement Learning

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Problem. Chapters 2–8 worked almost entirely inside two-player zero-sum games, where a Nash strategy secures a value v^* against anyone and CFR provably converges to it. Chapter 9 is the pivot into the multi-agent world, where the defining difficulty is **non-stationarity**: every agent is learning at once, so from any one agent’s seat the environment (which contains the others) never holds still. This is the launch point for the thesis’s multi-agent contributions — dynamic opponent modeling (#1), safe exploitation without a minimax anchor (#2), and population-level evaluation (#3).

Approach. Build and compare a control and the field’s four structural answers to non-stationarity on small, *exactly-solvable* testbeds, so every learner is graded against ground truth. **Independent learning** (the control that fails) on four canonical matrix games; **CTDE** — centralize the critic at training, decentralize the actor at execution — via MADDPG and MAPPO on cooperative tasks; **PSRO** — a game played over a *population* of policies (meta-Nash + best-response oracle), reusing Chapter 7’s exact best response as both oracle and exploitability metric — on Kuhn, Leduc, a matrix game, and a native Goofspiel; **learned communication** (CommNet); and **LOLA** (differentiate through the opponent’s learning step) on the Iterated Prisoner’s Dilemma. **All numbers below are measured.**

Key results (measured).

- *Independent learning fails exactly where theory says.* It converges to Nash on Prisoner’s Dilemma (defection), Stag Hunt (the risk-dominant corner), and Battle of the Sexes, but **Matching Pennies never converges** — its distance-to-Nash actually *grows* ($0.30 \rightarrow 0.48$), a sharper-than-predicted non-convergence (a contradicted prediction, §9 of the report).
- *PSRO is the game-theory $\leftrightarrow$ MARL bridge, and it works on the small games.* Exploitability $\rightarrow$ machine zero on **Kuhn** ($0.917 \rightarrow 2 \times 10^{-16}$, 6 rounds), $\rightarrow 0$ on the matrix game and Rock–Paper–Scissors. Self-play confirms the mechanism: on Kuhn the **average**-iterate NashConv falls $0.24 \rightarrow 0.031$ while the **last** iterate keeps oscillating — why a population/averaging is needed.
- *A centralized critic fits its value target almost exactly.* On CoopSignal the centralized critic’s residual is $3.2e-11$ vs the independent critic’s 0.077 — a far better fit of the value target (not the same as lower policy-gradient variance).
- *Communication clears the guessing ceiling.* CommNet with the channel ON scores 0.795 vs 0.204 OFF (ceiling $1/K = 0.2$) — a learned, not designed, protocol.
- *LOLA turns defectors into cooperators.* IPD per-step return rises from 1.04 (naive defection) to 2.82 (LOLA cooperation); zeroing the look-ahead recovers the naive gradient exactly (the mechanism check).
- *Honest negatives (kept predictions, §9 of the report).* PSRO on **Leduc** declines but hits a scaling wall ($4.75 \rightarrow 2.16$ after 20 rounds, not < 0.5); **Goofspiel $K=4$** oscillates rather than converging (a flagged code anomaly, documented not fixed); and on the **climbing game** no method reaches the optimum 11 (IL/MAPPO 7; MADDPG, as a COMA-style variant, 5) — a centralized critic fits its target better but does not by itself solve hard-exploration coordination.

Thesis connection. PSRO is Chapter 2’s iterated best response lifted to a population and the empirical backbone here; LOLA is *dynamic* opponent modeling, the moving-target complement to Chapter 7’s static read (Contribution #1); PSRO’s meta-game is a population-level evaluation methodology (Contribution #3). The Leduc scaling wall echoes Chapter 8’s global-vs-local finding, and the place where every two-player guarantee stops — the **missing $N>2$ minimax anchor** — is exactly Contribution #2’s problem statement.

Open questions. Can an approximate (RL) oracle push Leduc PSRO close to Nash, and at what cost to the guarantee? What breaks the Goofspiel- $K=4$ oscillation (de-duplication? a mixed population? a different meta-solver?), and why does a centralized critic *hurt* on the climbing game? And underneath all of it: with no minimax value for $N>2$, what does “safe” mean for a population, and can PSRO’s meta-game supply the substitute (Contribution #2)?

Chapter 10 One-Pager — Population-Based Training and Evolutionary Game Theory

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Problem. Self-play can go in circles: in a cyclic game (rock-paper-scissors) “get better” has no meaning, so a population that only trains against itself spins instead of improving. Chapter 10 is the population-level layer of the thesis — how to *diagnose* whether a game/population is a skill ladder or a wheel of counters, and how to *train and evaluate* a whole population of agents. It is the launch point for automated opponent modeling (#1), population safe-exploitation without a minimax anchor (#2), and population-level evaluation (#3).

Approach. Two halves, both graded against exact references. **Part I — evolutionary game theory as a diagnostic:** replicator dynamics on four solvable matrix games (checked against analytic ESS/Nash) and the transitive/cyclic **spinning-top** decomposition. **Part II — an AlphaStar-style PBT league** of neural PPO agents on Leduc Hold'em (three agent types: main / main-exploiter / league-exploiter, plus freezing and PFSP), evaluated with **EGTA/meta-Nash**, Elo, and diversity metrics — every neural policy extracted to a *tabular* policy so Chapter 7's **exact** best response measures its exploitability. **All numbers below are measured.**

Key results (measured).

- *Replicator dynamics reproduce every analytic outcome.* Prisoner's Dilemma → all-Defect; Hawk-Dove → the interior 0.5 ESS (orbit radius 0.0); **Rock-Paper-Scissors never converges** (orbits the centre, radius 0.095); Stag Hunt → a basin-dependent pure ESS.
- *The spinning-top diagnostic works, and Leduc's structure depends on the population.* Hodge decomposition gives RPS transitive 0.0 / cyclic 1.0 and a skill ladder 1.0 / 0.0 (the SVD rank-1 method wrongly gives RPS 0.707). The **PSRO best-response** meta-game on Leduc is mostly **cyclic** (≈ 0.45 transitive, 27 three-cycles); the **league snapshot** meta-game is mostly **transitive** (≈ 0.94 -0.98) — a contradicted prediction, §3 and §8.2 of the report.
- *The league produces strong individuals.* Its best frozen snapshot reaches exploitability (NashConv) 1.305 (scale), beating exact PSRO (2.163) and matching self-play's best iterate (1.396; its final agent is 3.683) — one run each; the CFR-Nash floor is 0.0099.
- *Honest negatives (kept predictions).* League exploitability is **non-monotone at scale** ($4.73 \rightarrow \min \approx 1.21 \rightarrow 2.05$ over 120 epochs) — the live agents regress late; the best agents are frozen snapshots. And the **meta-Nash mixture is more exploitable than the population's best member** at scale ($3.418 > 1.305$) — meta-Nash minimizes meta-game regret, not full-game exploitability, and the meta-Nash gave zero weight to the least exploitable agent. Diversity is thin (participation ratio 1.9, a single behavioral cluster).

Thesis connection. The league is Chapter 9's PSRO made asynchronous with neural oracles, reusing Chapter 7's exact best response as the yardstick; main exploiters are automated opponent modelers (Contribution #1); EGTA/meta-Nash is the population evaluation methodology (Contribution #3). The league's missing guarantee — it can regress, and its mixture can be exploitable — is the population form of the **missing $N > 2$ safety anchor** (Contribution #2). The transitive/cyclic diagnostic predicts Chapter 11's FFA coalition games will be strongly cyclic.

Open questions. Does best-snapshot retention / population regularization remove the late regression (a training fix) or is it inherent to the three-type design (a design fix)? Should a population ship a selected best-response-robust member or a diversity-regularized meta-solver instead of the meta-Nash mixture? And underneath both: with no minimax value for a population, what does “safe” mean, and can the exploiter mechanism be given a guarantee rather than staying heuristic (Contribution #2)?

Chapter 11 One-Pager — Coalition Formation and Detection

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Problem. Chapters 2-10 all leaned on one crutch: a two-player game with an exact best-response and exploitability oracle, so “did it work?” was one number. Add a third player and alliances become possible — form, exploit, betray — while Nash turns both intractable and guarantee-free, since it ignores coalitions entirely. Chapter 11 removes the crutch deliberately. This is the thesis frontier, not a consolidation step.

Approach. A native 4-player **So Long Sucker** engine — the bargaining game of Hausner, Nash, Shapley and Shubik, published in 1964 — implemented from the published rules, with a 2-player **minimax endgame** as its only exact anchor and the De Carufel & Jerade endgame analysis as the (not yet cross-checked) reference. On top: a **coalition detector** (help/harm from chip placement), **Shapley credit** redefined as the value of a member’s win probability, a **coalition-aware MAPPO** trainer blending coalition and winner-takes-all reward by a weight **alpha**, and **EGTA + spinning-top** analysis reused from Chapter 10. **All numbers below are measured**, with one caveat: **scale_results.json** is a **pre-fix** run cited only as evidence of the bug below — the authoritative figures are **sweep_scale.json** (5 seeds) and **post-fix smoke_results.json**.

Key results (measured).

- *Certified wherever exactness exists.* 100/100 random games terminate, all rewards are zero-sum, **endgame_mismatches:** 0 against exact minimax. The detector recovers a **planted {0,1} alliance** from the move stream alone (pair score **10.0**, cross-pair entries 0 or -1; a sanity check on one scripted, overt pair), and Shapley reproduces the glove and majority toys exactly, core emptiness included.
- *The chapter’s biggest finding is a bug, and its lesson outlived it.* Two red checks shared one cause: **~99.5% of random games in this engine end in deadlock**, so the winner falls to **_most_chips**, whose lowest-index tie-break gave seat 0 twice its fair share (all-random winners [94,42,33,31]). With an unbiased tie-break, symmetric Shapley spread fell **0.54 -> 0.013** and winners went uniform; the same fix deflated an impressive **0.87** hero win rate to an honest **~0.41** against 0.25 for random play. In a game that nearly always ends near-tied, the tie-break rule is load-bearing.
- *“No effect at scale” was overturned by finding the right knob — with a caveat.* **alpha** dominates and the **0.3 default sits in a dead zone**: at **alpha ~ 0** the proxy-credit gap is **+0.0376 +/- 0.0103** (the Shapley agents’ score is ~4.5x the sparse **0.0109**), while **every scale-tier alpha >= 0.3 cell is negative**. The effect is larger in the scale tier (12x the smoke tier at the same synergy; game size and training length change together), and the cheap critic-value proxy beats the rollout credit (**+0.038 vs +0.013**). But as implemented, neither credit encodes who helped whom, and the coalition score also counts hostile interactions: a dense reward raises the score, which is not yet evidence of learned coalitions.
- *The coalition score is bought, not free.* **alpha = 0** drops hero win rate to **~0.29**, close to random play (0.25), while **alpha >= 0.1** holds **~0.52**.
- *Strongly cyclic, honestly short of the bar.* The skill-ladder pool is transitive-dominant (cyclic **0.25-0.31**), but a coalition pool pushes cyclic to **~0.57-0.69** — large, yet still failing the strict “>50% dominance” rule, so the check stays red. Harness: **4/5 PASS**.

Thesis connection. Contribution #1 generalises cleanly: opponent modelling lifts from “what kind of player is this” to “who is allied with whom”, read straight off the moves. Contribution #2 gets its problem statement sharpened rather than solved — with no minimax value and an **empty core**, “safe” cannot mean bounded deviation from equilibrium; a behavioural anchor in the spirit of piKL is a candidate, not a guarantee. Contribution #3 gains a working substitute for exploitability in EGTA plus the spinning-top decomposition, inheriting Chapter 10’s caveat that the population you assemble decides whether you see a ladder or a wheel.

Open questions. Reconciling the engine’s turn and tie-break model against De Carufel & Jerade is the highest-value follow-up — both the seat-0 bug and the red cyclic check trace to documented simplifications, and the paper’s theorems are not yet checked. Whether a 3- or 4-player-aware EGTA projection surfaces the cycling the 2-type collapse discards. Whether a credit built on the detector’s help/harm matrix, tested against a coalition-free dense reward, produces real coalitions. And what “safe” can mean at all when the core is empty.

Chapter 12 One-Pager — Sequence Models and LLM Agents in Strategic Settings

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Problem. Two post-classical ways to play a game bypass everything Chapters 2-11 built. **Sequence modelling** recasts play as conditional prediction — feed a transformer (**return-to-go**, **state**, **action**) triples and ask for the next action given a desired return, no value function, no self-play — and **ARDT** adds minimax relabelling for worst-case robustness. **LLM agents** skip training entirely: hand a model the rules in English. Chapter 12 scores both against an exact benchmark.

Approach. Kuhn Poker, where Chapter 2 supplies exact Nash and exact exploitability, with a narrow port to Leduc Hold'em as a complexity check. Compared: Nash-CFR, behavioural cloning, a Decision Transformer, ARDT, and four LLM agents (a scripted offline stub with no language model; gpt-oss-20b, Qwen2.5-7B and OpenThinker3-7B, run locally in LM Studio on an RTX 5090 at temperature 0.7), plus follow-ons asking *why* that ranking falls out. **All numbers below are the author's own measurements**, from committed JSON, with two caveats: two results were **retracted** mid-session (an impossible +1.59 gap closed, and a Leduc -0.83 from a decoder discarding 70% of the probability mass), and the Nash reference is 0.0162 chips at 5,000 CFR iterations but 0.0061 at 50,000 — both ~0.

Key results (measured).

- *Plain cloning beats DT and ARDT.* **BC 0.055 < ARDT 0.469 < DT 0.799** against Nash **0.0162**; the real LLMs score **0.25–0.33** (the scripted offline stub: 0.833). Cloning lands within 0.04 chips of Nash by copying a near-Nash policy, while the DT routes that same policy through a return-conditioning channel carrying mostly luck — conditioning actively destroys information here.
- *Return conditioning never steers, on either game.* **high(+2) = 0.6981** vs **low(-2) = 0.6928**: tied, and ordered wrong. The Kuhn explanation (a 4-value payoff alphabet) was **half-refuted** on Leduc: 15 payoff values made the predicted artefact vanish, but steering still failed to appear (**r** = +0.062).
- *The models play better than they can explain.* The hypothesis going in was “knows the right frequency, cannot sample it”; the data says the reverse. Scored as strategies, *stated* frequencies are far worse than *played* ones — Qwen **0.921** vs **0.357** chips, gpt-oss **1.576** vs **0.392**. Probe behaviour; do not ask.
- *Exploitative by default, not adaptive.* Against the one opponent with enough statistical power (AlwaysPass), the model (Qwen2.5-7B, 120 hands) captures **83%** of available exploitation from the first half onward and does not improve; against the other two the error is ±0.53–0.80. Against the bot zoo it wins **0.177 chips/hand** vs **0.110** for Nash at an exploitability of 0.357 vs 0.006, with nearly 90% of the advantage coming from the passive and random opponents.
- *Model size does not explain the ranking, and the LLM edge belongs to Kuhn.* **Qwen2.5-7B beats gpt-oss-20b (3.6B active parameters) by +0.162 chips/hand** over 20k hands per pair, strictly transitively. On Leduc the LLM is indistinguishable from the DT (-0.463 vs -0.454) and loses on average to weak opponents (-0.071 where Nash gets +0.582; it wins only against the two passive types), with **100%** of its illegal-action mass a single misconception: folding when checking is free.
- *One scalar hides the diagnosis.* A single Queen node is **41.4%** of total exploitability, while value-betting the King at 1.00 where the CFR equilibrium mixes at 0.68 costs **0.1%** — deviation size and cost are nearly uncorrelated. Harness: **3 PASS / 2 FAIL**, the failures left as they are.

Thesis connection. Contribution #1 gets a negative result worth having: adaptation from in-context history does not appear (told the opponent's type, the models do respond), so an explicit opponent model must be built rather than assumed. Contribution #2 gets its trade-off measured on one plot — more exploitation (0.177 vs 0.110 chips/hand) at far higher exploitability (0.357 vs 0.006), mostly against weak opposition. Contribution #3 gets the per-decision decomposition and the measurement protocol.

Open questions. The most likely fix is ARDT's $Q(s,a)$ (not yet tested; with chance events such as the deal, ARDT's minimax target is no longer exact): a state-only return estimator cannot tell “this state is bad” from “*this action* is bad”, probably why the tau sweep contradicts theory and exploitability comes out *lowest* on the optimistic side — the default was left at the theoretically correct value rather than switched to buy a better number. Whether the Leduc misconception is one prompt line away. And the standing caveat: every Leduc LLM conclusion rests on one model, one prompt style, 600 hands.